

11th June 2026

Lab 1: HTTP Fundamentals & First FastAPI Endpoints

This builds directly on your Lab 0 setup. You learn HTTP methods, status codes, and build multiple API endpoints.

Prerequisite: Lab 0 completed (UV installed, FastAPI running, 'http://127.0.0.1:8000/docs' accessible)

By the end of this lab, you will be able to:

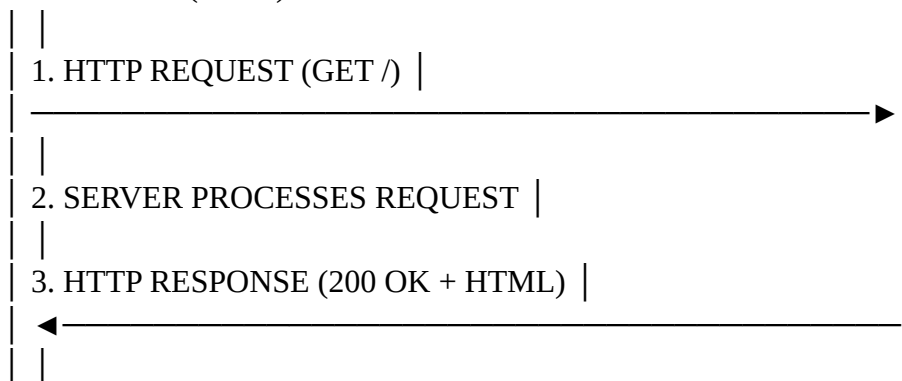
1. Explain HTTP methods (GET, POST, PUT, DELETE) and status codes
2. Create multiple API endpoints in FastAPI
3. Use path parameters and query parameters
4. Test your API using Swagger UI (`/docs`)
5. Understand the difference between path and query parameters

Part 1: HTTP Fundamentals (The 'Language' of the Web)

1.1 The Request-Response Cycle

When you type 'https://www.safaricom.co.ke' in your browser:

Your Browser (Client) Safaricom Server



1.2 HTTP Methods (The 'Verbs' of the Web)

Method	Purpose	Example	Analogy
--------	---------	---------	---------

-----	-----	-----	-----
-------	-------	-------	-------

GET	Retrieve data	'Show my M-PESA balance'	Asking a question
---------	---------------	--------------------------	-------------------

POST	Create new data	'Send money to 07xxx45678'	Submitting a form
----------	-----------------	----------------------------	-------------------

PUT	Update entire record	'Change my PIN to 1234'	Replacing entirely
---------	----------------------	-------------------------	--------------------

PATCH	Partial update	'Update my email only'	Editing a field
-----------	----------------	------------------------	-----------------

| **DELETE** | Remove data | 'Delete my account' | Throwing away |

1.3 HTTP Status Codes (The Server's 'Emotions')

| Code Range | Meaning | Example |

|-----|-----|-----|

| **200-299** | Success (OK!) | '200 OK' - Request succeeded |

| **201** | Created | New resource created successfully |

| **400-499** | Client Error (Your fault) | '404 Not Found' - Wrong URL |

| **500-599** | Server Error (Their fault) | '500 Internal Server Error' |

Memory Aid:

- 2xx = **2** Good (Success)

- 4xx = **4** Your fault (Client error)

- 5xx = **5** Server's fault

Part 2: Your First FastAPI Endpoints

2.1 Review: Your Lab 0 Code

Open your 'main.py' file. It should look like this:

```
python
```

```
from fastapi import FastAPI
```

```
app = FastAPI(title= 'My Backend API')
```

```
@app.get("/")
```

```
def root():
```

```
    return {'message': 'Hello from CIT Backend Course!'}
```

Run it:

```
uv run uvicorn main:app --reload
```

Test it: Open <http://127.0.0.1:8000/docs>

2.2 Adding a GET Endpoint with Path Parameters

Path Parameters are variables embedded in the URL path.

Example URL: /users/123 (where 123 is the user ID)

```
@app.get('/users/{user_id}')
```

```
def get_user(user_id: int):  
    return {'user_id': user_id, 'name': f'User {user_id}'}
```

Test in Swagger UI:

1. Go to /docs
2. Click GET /users/{user_id}
3. Click "Try it out"
4. Enter 5 as user_id
5. Click "Execute"

Response:

```
{  
  "user_id": 5,  
  "name": "User 5"  
}
```

2.3 Adding a GET Endpoint with Query Parameters

Query Parameters come after ? in the URL.

Example URL: /items?limit=10&category=electronics

```
@app.get("/items")
```

```
def list_items(limit: int = 10, category: str = "all"):  
    return {"limit": limit, "category": category, "items": []}
```

Test URLs:

- http://127.0.0.1:8000/items → uses defaults
- http://127.0.0.1:8000/items?limit=5 → limit=5, category=all
- http://127.0.0.1:8000/items?limit=20&category=books

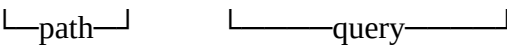
2.4 Path vs. Query Parameters: The Difference

Feature	Path Parameter	Query Parameter
Location	Part of URL path	After ?

Feature	Path Parameter	Query Parameter
Syntax	/users/{id}	/users?role=admin
Purpose	Identify a specific resource	Filter, sort, paginate
Example	"Get user with ID 5"	"Get first 10 users"
Required?	Usually yes	Usually optional

Illustration:

<https://api.example.com/users/123/posts?limit=5&sort=desc>



 "Which user?" "How many? Which order?"

Part 3: Student Exercises

Exercise 1: Build a "Welcome" Endpoint

Task: Create a GET endpoint at /welcome/{name} that returns:

```
{"message": "Welcome, Alice!"}
```

Test: Call /welcome/John → should return {"message": "Welcome, John!"}

Exercise 2: Build a "Calculator" Endpoint

Task: Create a GET endpoint at /calculate that accepts three query parameters:

- a (float, required)
- b (float, required)
- operation (string, default = "add")

Requirements:

- If operation = "add", return {"result": a + b}
- If operation = "subtract", return {"result": a - b}
- If operation = "multiply", return {"result": a * b}
- If operation = "divide", return {"result": a / b} (handle division by zero)

Test URLs:

- /calculate?a=10&b=5&operation=add → {"result": 15}
- /calculate?a=10&b=5&operation=multiply → {"result": 50}

Exercise 3: Create a "Product" API

Task: Create a simple in-memory product catalog using a Python list.

Add this after your app definition

```
products = [  
    {"id": 1, "name": "Laptop", "price": 75000},  
    {"id": 2, "name": "Mouse", "price": 1500},  
    {"id": 3, "name": "Keyboard", "price": 3500},  
]
```

Create these endpoints:

Endpoint	Method	Description
/products	GET	Return ALL products
/products/{product_id}	GET	Return a single product by ID (404 if not found)
/products/search	GET	Search by name (query param q)

Example responses:

- GET /products → returns the entire list
- GET /products/2 → returns {"id": 2, "name": "Mouse", "price": 1500}
- GET /products/99 → returns {"error": "Product not found"}
- GET /products/search?q=lap → returns [{"id": 1, "name": "Laptop", "price": 75000}]

Exercise 4: Create a POST Endpoint

Task: Create an endpoint to add a new product to the catalog.

```
from fastapi import FastAPI
```

```
from pydantic import BaseModel
```

Define the expected request body shape

```
class ProductCreate(BaseModel):
```

```
    name: str
```

```
    price: float
```

```
@app.post("/products")
```

```
def create_product(product: ProductCreate):
```

```
    new_id = len(products) + 1
```

```
    new_product = {"id": new_id, "name": product.name, "price": product.price}
```

```
products.append(new_product)

return {"message": "Product created", "product": new_product}
```

Test with Swagger UI (/docs):

1. Go to POST /products
2. Click "Try it out"
3. Enter: {"name": "Monitor", "price": 12000}
4. Execute

Expected response:

```
{
  "message": "Product created",
  "product": {"id": 4, "name": "Monitor", "price": 12000}
}
```

Exercise 5: Challenge – Port Switching & Environment Variables

Scenario: Port 8000 is sometimes blocked on university Wi-Fi.

Task:

1. Stop your server (Ctrl+C)
2. Restart on a different port: `uv run uvicorn main:app --reload --port 9000`
3. Verify Swagger UI works at `http://127.0.0.1:9000/docs`

Bonus: Make the port configurable using an environment variable.

Windows PowerShell

```
$env:PORT=9000
```

```
uv run uvicorn main:app --reload --port $env:PORT
```

Linux/macOS

```
export PORT=9000
```

```
uv run uvicorn main:app --reload --port $PORT
```

Part 5: Key Takeaways

Concept	Summary
GET	Retrieve data (Read)
POST	Create new data (Create)
Path params	/users/{id} – identifies specific resource
Query params	/users?role=admin – filters/sorts/paginates
Status codes	200 = OK, 404 = Not found, 500 = Server error
Swagger UI	Automatic interactive documentation at /docs